MyCodeSchool

Lecția 2 — Workflows: orchestrarea scrisă de tine

teorie scurtă · 6 exerciții care construiesc un singur workflow · 2 provocări · pe platforma ta din `ms\`

Teza lecției. Cea mai scumpă greșeală pe orice platformă nu e linia greșită, ci **fixul pus într-un singur loc din trei**: repari linia, nu regula. Nu e neatenție — după primul fix, mintea consideră problema închisă și nu mai întreabă „unde mai e?". În fișa 2 (ex. 7 și 14) ai pus întrebarea la plural cu un skill: mergea, cât timp îți aminteai să-l chemi pe toate. Un workflow o pune la plural **mecanic**: un agent pe fiecare caz, scris dinainte în script. Nu depinde de faptul că își amintește cineva de „celelalte două".

Terenul: atelierul tău, `C:\Projects\teste claude\ms\`, cu `CLAUDE.md` și `.claude\` din fișele 1–2. Gardul din `settings.json` rămâne cum e. Înainte de exercițiul 1, verifică un singur lucru: **toate repo-urile platformei sunt clonate în `ms\`** (fișa 2, ex. 1–2). Un agent nu poate citi un repo care nu e pe disc — și nici nu-ți va spune că lipsește, decât dacă îl întrebi.
Totul e read-only. Workflow-urile din lecție doar citesc repo-urile; nu scriu fișiere, nu ating clusterul, nu fac commit. Ce găsesc, repari tu, prin git, ca până acum.

Costul, înainte de primul exercițiu. Un workflow pornește zeci de agenți, fiecare cu contextul lui. Cazul din teoria de mai jos: 19 agenți, ~2,5 minute, ~965k tokeni — pentru un singur commit de ~250 de linii. Socoteala de buzunar: **~50k tokeni pe agent**. Pornește doar când ceri explicit, iar progresul pe faze și agenți îl urmărești cu `/workflows`. La final primești o notificare cu numărul de agenți, tokenii și durata — noteaz-o de fiecare dată, îți trebuie la provocarea 2.

Teoria — mecanismul, apoi de ce

T1. Subagent sau workflow: cine decide

	Subagent (Ghid pag. 7)	Workflow
Cine îl pornește	modelul, când crede că e nevoie	tu, explicit; scriptul pornește apoi agenții
Câți agenți, în ce ordine	decide modelul pe parcurs	scrie în script, dinainte: cine lucrează în paralel, cine verifică
Ce face cu rezultatul	îl citește și improvizează	cod: `filter`, numără, deduplică, decide pasul următor
Când se oprește	când i se pare gata	când se termină scriptul

Subagentul e un coleg pe care modelul îl cheamă. Workflow-ul e **organigrama**: o scrii tu, o dată, și se execută la fel de fiecare dată.

T2. Cum pornește și unde stă

- Tool-ul se cheamă `Workflow` și rulează un script **JavaScript** (nu TypeScript: `: string[]` nu se parsează) în fundal.
- Pornește **doar la cerere explicită**: scrii „use a workflow", „rulează workflow-ul X" sau cuvântul cheie `ultracode`. Altfel modelul lucrează singur, ca până acum.
- Scripturile salvate stau în `.claude\workflows\<nume>.js` și se rulează după nume. Primele cinci exerciții țin scripturile într-un folder de lucru, `ms\wf\`; abia la exercițiul 6 îl salvezi pe cel bun.

- Agenții din workflow primesc același `CLAUDE.md` ca sesiunea ta — harta ta e citită de fiecare agent. Nu le-o mai repeta în prompt.

T3. Piesele scriptului

Piesa	Ce face
`export const meta = { name, description, phases }`	obligatoriu, primul. **Obiect literal pur** — fără variabile, apeluri sau `` `${...}` ``: se citește înainte să ruleze scriptul (în dialogul de permisiune și în lista de workflow-uri)
`agent(prompt, opts)`	pornește un subagent. Fără `schema` întoarce text; cu `schema` (JSON Schema) întoarce obiectul, deja validat. Întoarce `null` dacă agentul e sărit sau moare — de aici `.filter(Boolean)` peste tot
`opts`	`label` (numele în `/workflows`), `phase`, `schema`, `model`, `effort` (`'low'` … `'max'`), `agentType`, `isolation: 'worktree'` — ultima doar când agenții **modifică** fișiere în paralel; în lecția asta, niciodată
`parallel([() => agent(...), ...])`	rulează toate funcțiile odată și **așteaptă toți** — e o barieră. Cele eșuate devin `null`
`pipeline(elemente, pas1, pas2, ...)`	fiecare element trece prin pași **independent**, fără barieră între pași. Pasul primește `(rezultatAnterior, elementOriginal, index)`. Implicitul pentru lucru în mai mulți pași
`phase(titlu)` · `log(mesaj)` · `args`	grupare în progres · un rând pentru tine · input-ul dat la pornire, ca JSON real

T4. Pipeline sau barieră

Trei auditori, fiecare urmat de verificatorii lui. Aceeași muncă, două orchestrări:

```
bariera (parallel, apoi parallel):
  audit A  ====                 | verif A --
  audit B  ============         | verif B ---
  audit C  ======               | verif C --
                                ^ abia aici pornesc toti verificatorii

pipeline:
  audit A  ==== verif A --
  audit B  ============ verif B ---
  audit C  ====== verif C --
```

Cu barieră, durata e suma celor mai lenți de pe fiecare etapă; cu `pipeline`, e cel mai lent lanț. **Bariera e justificată doar când pasul următor are nevoie de toate rezultatele odată:** deduplicare peste tot setul, oprire dacă s-a găsit zero, comparație între rezultate. „Trebuie să fac un `map` / `filter` înainte" **nu** o justifică — transformarea intră într-un pas din `pipeline`.

T5. Ce e interzis în script, și de ce

`Date.now()`, `Math.random()`, `new Date()` fără argument aruncă eroare. Motivul e **reluarea**: cu `resumeFromRunId`, scriptul rulează din nou de la început, iar **cel mai lung prefix neschimbat** de apeluri `agent()` vine din cache, instant. De la primul apel modificat încolo, **tot** rulează live. Un prompt care conține ora ar fi „modificat" la fiecare rulare, deci cache-ul n-ar prinde nimic. Tot de aceea scriptul n-are acces la fișiere sau la Node: **agenții citesc, scriptul doar orchestrează**.

T6. Un caz real: review → verificare

Un commit de elev, ~250 de linii: o entitate `User` și testul ei. Workflow-ul: trei revieweri pe dimensiuni (compilare + teste · modelul JPA · igiena repo-ului), în `pipeline`; fiecare problemă găsită merge la un **sceptic**, care încearcă s-o infirme.

Agenți	Durată	Tokeni	Găsite	Confirmate	Respinse	Distincte
19	~2,5 min	~965k	16	14	2	**9**

Problema roșie era o aserțiune care nu verifica nimic:

```
assertThat(user.equals(copie));
assertThat(user.equals(copie)).isTrue();
```

Prima linie doar **construiește** aserțiunea; fără `.isTrue()` nu se verifică nimic, deci testul trece mereu. Scepticul a dovedit-o prin mutație, pe o copie: a pus o așteptare greșită și a obținut `BUILD SUCCESS`. Trei lecții:

- **Lipsea o barieră.** Aceeași aserțiune a fost găsită de două dimensiuni și verificată de două ori. O deduplicare pe `file:line` **înainte** de verificare — singurul loc unde bariera e corectă — ar fi tăiat 16 verificări la 9.
- **Costul trebuie să se potrivească cu ținta.** Pentru un commit mic ajunge un singur review. Workflow-ul merită la lucru mare: un audit pe cinci repo-uri, un modul întreg.
- **Verificarea a meritat.** Problema roșie a fost dovedită prin rulare, nu afirmată, iar două alarme false au fost eliminate înainte să ajungă la om.

T7. Tiparele

Tipar	Ce face	Unde
Verificare adversarială	un sceptic pe fiecare problemă, care încearcă s-o **infirme**; la îndoială, o respinge	ex. 3–4
Panel de judecători	perspective diferite pe aceeași întrebare, apoi judecători independenți care le compară; decide majoritatea	ex. 5
Loop-until-dry	runde de căutare până când K runde la rând nu mai aduc nimic nou	provocarea 1
Critic de completitudine	un ultim agent care întreabă „ce n-a fost citit, ce n-a fost verificat?"	opțional, oriunde

Cum se citește partea de exerciții. Exercițiile 1–3 introduc noțiuni noi, deci primești scriptul exact și ce trebuie să vezi. De la 4 încolo primești doar sarcina, un **gata când**, un **indiciu** (citit după prima încercare) și linia de **teorie**. Fiecare exercițiu pornește de la scriptul celui de dinainte. *Ghid* = `GHID-CLAUDE-CODE.pdf` · *Cuvinte* = `CUVINTE-CHEIE.pdf` · *F2* = fișa 2 · *L1* = lecția 1 · *Programa* = programa DevOps.

Partea I — Primele piese, cu scriptul dat

Un singur agent, care întoarce text

1

Creează `ms\wf\01.js`, exact așa:

```
export const meta = {
  name: 'ms-pipeline',
  description: 'Job-urile din pipeline-ul fiecarui serviciu',
}

const raport = await agent(
  'Pentru fiecare serviciu din ms (data, importer, client) gaseste ' +
  'fisierul de pipeline din .github/workflows si listeaza job-urile ' +
  'lui, in ordine, cu fisier:linie. Nu modifica niciun fisier.',
  { label: 'pipeline-uri' }
)
return raport
```

Apoi, în Claude, pornit din `ms\`:

```
Use a workflow: ruleaza exact scriptul din wf/01.js, fara sa-l modifici.
```

Cât rulează, deschide `/workflows`.

Gata când: în `/workflows` ai văzut **un singur agent**, `pipeline-uri`; rezultatul e text, câte un job pe rând, cu `fisier:linie` — pentru **trei** fișiere de pipeline, nu două; iar `git status` e curat în toate repo-urile. Dacă Claude ți-a „îmbunătățit" scriptul înainte să-l ruleze, cere-i să ruleze exact fișierul — ce rulează e ce ai scris tu.

Teorie: T2, T3 · Ghid pag. 7 — Piesa 5, subagentul: aici e același agent, doar că îl pornește scriptul, nu modelul.

Același agent, cu schemă: un rezultat pe care scriptul îl poate număra

2

Textul de la exercițiul 1 îl citești tu; scriptul nu poate face nimic cu el. Cu `schema`, agentul e obligat să întoarcă un obiect validat. Creează `ms\wf\02.js` din trei bucăți, în ordinea asta.

```
export const meta = {
  name: 'ms-probleme-ci',
  description: 'Problemele din pipeline-urile ms, ca lista structurata',
}
```

```
const PROBLEME = {
  type: 'object',
  properties: {
    probleme: {
      type: 'array',
      items: {
        type: 'object',
        properties: {
          file: { type: 'string' },
          line: { type: 'integer' },
          prioritate: { type: 'string', enum: ['rosu', 'galben', 'verde'] },
          descriere: { type: 'string' },
          amprenta: { type: 'string' },
        },
        required: ['file', 'line', 'prioritate', 'descriere'],
      },
    },
  },
  required: ['probleme'],
}
const FARA_VALORI =
  'Nu scrie niciodata o valoare sensibila. Pentru ea completezi ' +
  'amprenta: primele 4 caractere si lungimea, de forma ab12/40.'
```

```
const r = await agent(
  'Auditeaza pipeline-urile celor trei servicii din ms: pasi care ' +
  'lipsesc la unul si exista la altul, valori scrise in clar, repo-ul ' +
  'in care scriu. Pentru fiecare problema: fisierul (cale din ms/), ' +
  'linia, prioritatea, o propozitie. Nu modifica niciun fisier. ' +
  FARA_VALORI,
  { label: 'audit CI', schema: PROBLEME }
)
if (!r) return 'agentul nu a raspuns'
return {
  total: r.probleme.length,
  rosii: r.probleme.filter(p => p.prioritate === 'rosu'),
}
```

`FARA_VALORI` nu e un moft: **repo-urile tale sunt publice**, iar raportul unui audit ajunge în terminal, în copy-paste, în note. Raportul nu scrie niciodată o valoare sensibilă — doar amprenta ei, ca s-o poți recunoaște în alt fișier.

Gata când: rezultatul arată ca forma de mai jos — un obiect, nu proză — și fiecare problemă cu valoare sensibilă are `amprenta`, nu valoarea:

```
{ "total": 5,
  "rosii": [ { "file": "<repo>/<fisier>", "line": 12,
               "prioritate": "rosu", "descriere": "...",
               "amprenta": "ab12/40" } ] }
```

Teorie: T3 — `schema` și `null` · F2 ex. 7 — aceeași comparație a pipeline-urilor, acolo ca tabel citit de tine · Programa: lecția 19, Secret scanning.

Pipeline: trei auditori, fiecare problemă la un sceptic

3

Trei dimensiuni care se uită la aceleași repo-uri din unghiuri diferite. Fiecare problemă găsită merge, imediat ce auditorul ei a terminat, la un agent care încearcă s-o **infirme**. Creează `ms\wf\03.js`: întâi `meta`,

```
export const meta = {
  name: 'ms-audit',
  description: 'Audit ms pe 3 dimensiuni, fiecare problema la un sceptic',
  phases: [{ title: 'Audit' }, { title: 'Verificare' }],
}
```

apoi `PROBLEME` și `FARA_VALORI` copiate de la exercițiul 2, apoi schema verdictului și restul:

```
const VERDICT = {
  type: 'object',
  properties: {
    confirmata: { type: 'boolean' },
    dovada: { type: 'string' },
  },
  required: ['confirmata', 'dovada'],
}
```

```
const DIM = [
  { nume: 'ci', ce: 'pipeline-urile celor trei servicii' },
  { nume: 'gitops', ce: 'cele doua repo-uri GitOps' },
  { nume: 'secrete', ce: 'valori sensibile in clar, in orice repo' },
]
```

```
const rezultat = await pipeline(
  DIM,
  d => agent(
    `Auditeaza repo-urile din ms doar pe dimensiunea: ${d.ce}. ` +
    'Fisierul il dai cu cale din ms/. Nu modifici nimic. ' + FARA_VALORI,
    { phase: 'Audit', label: d.nume, schema: PROBLEME }
  ),
  (r, d) => parallel((r ? r.probleme : []).map(p => () =>
    agent(
      `Incearca sa INFIRMI problema ${p.file}:${p.line} - ` +
      `${p.descriere}. Citeste fisierul. Daca nu esti sigur, ` +
      'confirmata=false. ' + FARA_VALORI,
      { phase: 'Verificare', label: `sceptic ${d.nume}`, schema: VERDICT }
    ).then(v => ({ ...p, dim: d.nume, verdict: v }))
  ))
)
```

```
const toate = rezultat.filter(Boolean).flat()
return {
  gasite: toate.length,
  confirmate: toate.filter(p => p.verdict && p.verdict.confirmata),
  respinse: toate.filter(p => p.verdict && !p.verdict.confirmata).length,
}
```

Gata când: în `/workflows` ai prins momentul în care un sceptic al unei dimensiuni lucra **în timp ce** auditorul alteia încă nu terminase — asta e `pipeline`, fără barieră. Rezultatul are forma `{ gasite, confirmate: [...], respinse }`. Notează din notificarea de final: agenți, tokeni, durată. Apoi uită-te la `confirmate`: există aceeași `amprenta` de două ori? Sau aceeași problemă, venită o dată de la `ci` și o dată de la `secrete`?

Teorie: T4 · T6 — exact forma cazului real, cu aceeași gaură · T7 — verificarea adversarială.

Partea II — Workflow-ul tău: doar indicii de aici încolo

Bariera de deduplicare — singura care se justifică

4

Transformă `03.js` în `04.js`: auditorii rămân cei trei, dar între ei și sceptici pui **o barieră** care strânge tot, grupează duplicatele și trimite la sceptici **o singură problemă pe grup**, cu toate locurile ei. Secretele se grupează pe **valoare** (amprentă), restul pe `file:line`. Apoi scrie, în două rânduri, de ce bariera stă **aici** și n-ar avea ce căuta între auditori sau după sceptici.

Gata când: `log`-ul arată `N gasite, M distincte` cu M < N; numărul de sceptici din `/workflows` e M, nu N; și fiecare rezultat confirmat are lista `locuri` completă. Compară tokenii cu rularea de la ex. 3.

Indiciu: auditorii trec pe `parallel` (bariera e chiar `await`-ul lui), apoi grupezi în cod obișnuit — e o `Map`, nu un agent:

```
const unice = new Map()
for (const p of gasite) {
  const loc = `${p.file}:${p.line}`
  const cheie = p.amprenta || loc
  const deja = unice.get(cheie)
  if (!deja) unice.set(cheie, { ...p, locuri: [loc] })
  else if (!deja.locuri.includes(loc)) deja.locuri.push(loc)
}
log(`${gasite.length} gasite, ${unice.size} distincte`)
if (unice.size === 0) return 'nimic de verificat'
```

Scepticul primește acum `p.locuri.join(', ')` în loc de un singur `file:line` și încearcă s-o infirme **în fiecare loc**. Aici stă tot rostul lecției: o valoare găsită într-un fișier și reparată acolo e, de regulă, încă în alte două — grupul ți le arată pe toate odată, iar fix-ul pe care îl faci după aceea e pe regulă, nu pe linie. De ce aici: deduplicarea are nevoie de **toate** rezultatele, deci nimic nu poate merge mai departe până nu termină toți. Între auditori nu e nimic de comparat, iar după sceptici nu mai e nimic de economisit.

Teorie: T4 — cele trei cazuri de barieră justificată · T6 — „16 verificări la 9" · F2 ex. 14 — `/secrete` v2, gruparea pe valoare · Programa: lecția 38, Managementul secretelor.

Panel de judecători pe lanțul de tag-uri

5

Regula pe care stă toată platforma: **tag-ul pe care îl scrie CI-ul fiecărui serviciu trebuie să fie exact tag-ul pe care îl citește ArgoCD.** Scrie `05.js` care o verifică din trei perspective independente — pipeline-urile CI, repo-ul GitOps citit de ArgoCD (pornind de la `bootstrap/root.yaml`), istoria git (cine a scris ultima valoare: ci-bot sau mâna ta) — apoi trei judecători care primesc toate cele trei rapoarte și dau, pe fiecare serviciu, verdictul *închis / rupt*. Decide majoritatea.

Gata când: rezultatul are un rând pe serviciu, iar fiecare judecător a scris **din ce fișier a citit fiecare valoare** — repo, cale, linie. Apoi poarta, ca la F2 ex. 6: schimbă cu mâna o cifră din tag-ul citit de ArgoCD, fără commit, rulează din nou; acel serviciu trebuie să iasă `RUPT`. Pune cifra la loc, `git diff` gol. Compară verdictul cu ce-ți spune `/lant`: dacă nu sunt de acord, cine a citit din fișierul greșit?

Indiciu: perspectivele întorc text (fiecare vede doar o jumătate a lanțului, deci n-are ce verdict să dea); judecătorii întorc o schemă cu un rând pe serviciu: `serviciu`, `scrieCI`, `citesteArgo` (amândouă de forma `repo/fisier: valoare`), `ultimulAutor`, `inchis`. Aici bariera e justificată de al treilea caz din T4: judecătorul **compară** rapoartele între ele.

```
const PERSP = [
  'pipeline-urile CI: in ce repo, fisier si cheie scrie fiecare tag',
  'ArgoCD: de la bootstrap/root.yaml, ce fisier si ce tag citeste',
  'git log: cine a scris ultima valoare a fiecarui tag, ci-bot sau om',
]
```

```
const dosar = opinii.join('\n---\n')
const voturi = (await parallel([1, 2, 3].map(i => () =>
  agent(
    `Esti judecatorul ${i}. Trei colegi au cercetat lantul de tag-uri ` +
    `din perspective diferite:\n${dosar}\nVerifica in fisiere. ` +
    'Lantul e inchis doar daca tag-ul scris de CI e exact cel citit ' +
    'de ArgoCD. scrieCI si citesteArgo = repo/fisier: valoare. ' +
    `Nume de serviciu exact: ${SERVICII}.`,
    { phase: 'Judecatori', label: `judecator ${i}`, schema: LANT }
  )
))).filter(Boolean)
```

Cerința „repo/fisier" din prompt e tot exercițiul: un tag care coincide nu dovedește nimic dacă cele două valori n-au fost citite din fișierele pe care chiar le folosesc CI-ul și ArgoCD. Numele de serviciu le dai exact, într-o constantă, altfel trei judecători scriu trei nume și majoritatea nu se mai poate număra.

Teorie: T7 — panelul · L1 și F2 ex. 5–6 — `/lant`, valoarea și direcția ei · F2 ex. 8 — „cine scrie valoarea de obicei" · Programa: lecția 21, CI/CD II — livrare fără chei.

Salvat, rulat după nume, reluat din cache

6

Mută `04.js` în `ms\.claude\workflows\audit-ms.js` și adaugă-i `args`: lista dimensiunilor de rulat; fără `args`, toate. Rulează:

```
Ruleaza workflow-ul audit-ms cu args ["secrete"].
```

Apoi editează **doar promptul scepticului** (de exemplu, cere-i să spună și de când e valoarea în repo, din `git log`) și reia:

```
Reia ultima rulare audit-ms cu resumeFromRunId, cu scriptul modificat.
```

La final, editează promptul **auditorului** și reia încă o dată.

Gata când: la prima reluare auditorul a venit din cache (instant) și doar scepticii au rulat live; la a doua, a rulat **totul** live. Scrie într-o frază de ce — și de ce un `Date.now()` în promptul auditorului ar fi făcut ca prima reluare să arate ca a doua.

Indiciu: filtrul pe `args`, pus după `DIM`; în restul scriptului folosești `ALESE` în loc de `DIM`:

```
const ALESE = Array.isArray(args)
  ? DIM.filter(d => args.includes(d.nume))
  : DIM
log(`dimensiuni: ${ALESE.map(d => d.nume).join(', ')}`)
```

`args` vine ca JSON adevărat, deci `["secrete"]` e o listă, nu un text. Reluarea refolosește **cel mai lung prefix neschimbat** de apeluri `agent()`: la prima editare prefixul e „auditorul", la a doua prefixul e gol — primul apel e chiar cel schimbat. `meta.name` trebuie să fie `audit-ms`, la fel ca fișierul.

Teorie: T2 — unde stau scripturile · T5 — reluarea și de ce sunt interzise ceasul și hazardul · Ghid pag. 5 — comanda cu `$ARGUMENTS` : `args` e același lucru, pentru un workflow.

Partea III — Provocări, fără indicii

Loop-until-dry pe secrete

P1

Un singur auditor ratează. Scrie un workflow care caută valori sensibile în runde: în fiecare rundă, trei căutători care caută **în moduri diferite** (după conținut, după tipul fișierului, în istoria git). Se oprește când două runde la rând nu aduc nicio amprentă nouă. Fiecare amprentă nouă trece prin sceptic, iar raportul rămâne fără nicio valoare.

Done când: `log` -ul arată fiecare rundă cu `noi=N` , ultimele două cu `noi=0` ; o amprentă respinsă de sceptic nu reapare în nicio rundă ulterioară; iar rezultatul final are, pe fiecare amprentă, toate locurile — inclusiv cele din istorie.

Când NU folosești un workflow

P2

Trei sarcini: (a) review pe ultimul tău commit din `client` ; (b) un audit complet pe cele cinci repo-uri înainte să prezinți platforma cuiva; (c) „ce tag rulează acum `data-service` ?". Pentru fiecare decizi: un prompt, o comandă sau un skill pe care le ai deja, sau un workflow — și **estimezi înainte** câți agenți și câți tokeni.

Done când: ai un tabel cu trei rânduri — sarcină, unealtă, agenți estimați, tokeni estimați — scris **înainte** de orice rulare; doar (b) e workflow; după ce îl rulezi, valorile din notificarea de final sunt în factor de 2 față de estimarea ta.

Ce urmează — lecția Spring AI. Aceleași idei, în Java, în aplicație: `agent()` + `schema` devine `ChatClient` cu `.entity(Clasa.class)` , scepticul devine un evaluator, `parallel` devine `CompletableFuture` . Serviciul nou se livrează prin lanțul tău GitOps, ca celelalte, iar cheia de API intră în cluster ca sealed-secret, nu în clar.